Assignment 1: Command Line & Documentation

Student Name: WuYuekai
Student ID: ZY2557209

Command Line | Markdown | Python | Matrix Multiplication

1. Project Overview

The objective of this assignment is to practice basic remote development skills, including command line operations, Markdown documentation, and Python programming.

In this project, I implemented a simple matrix multiplication program using Python. I also collected system configuration information from my computer and documented the implementation, execution command, and verification process in Markdown.

The main goals of this assignment are:

- Practice command line operations
- Write technical documentation using Markdown
- Implement matrix multiplication in Python
- Verify the correctness of the algorithm
- Publish the report on a personal website

2. System Configuration

This assignment was completed on a Windows computer using Git Bash. Some Linux-specific commands, such as `lscpu` and `free -h`, were not available in my environment. Therefore, PowerShell commands were used to collect part of the system information.

Item	Result
CPU Model	Intel(R) Core(TM) i7-14700HX
Memory Size	31.78 GB
Operating System	Microsoft Windows 11 家庭中文版
OS Version	Microsoft Windows [版本 10.0.26200.8246]
Git Bash System Info	MINGW64_NT-10.0-26200 kk 3.6.6-1cdd4371.x86_64
Compiler Version	GCC not installed
Python Version	Python 3.12.10

The commands used to collect the information included:

```
python --version
gcc --version
uname -a
cmd /c ver
powershell -Command "Get-CimInstance Win32_Processor"
powershell -Command "Get-CimInstance Win32_ComputerSystem"
powershell -Command "Get-CimInstance Win32_OperatingSystem"
```

3. Implementation Details

Matrix multiplication is a basic operation in linear algebra.

If matrix `A` has size `m × n` and matrix `B` has size `n × p`, then the result matrix `C` has size `m × p`.

Each element of the result matrix is calculated by multiplying one row of matrix `A` with one column of matrix `B` and summing the products.

The formula is:

```
C[i][j] = A[i][0] × B[0][j] + A[i][1] × B[1][j] + ... + A[i][n-1] × B[n-1]
```

```
[j]
```

The algorithm uses three nested loops:

Loop	Purpose
Outer loop	Selects the row of matrix A
Middle loop	Selects the column of matrix B
Inner loop	Calculates the sum of products

The time complexity of this basic matrix multiplication algorithm is `O(m × n × p)`.

For square matrices of size `n × n`, the time complexity is `O(n³)`.

4. Python Language Implementation

4.1 Source Code

```python
# matrix_multiply.py
# This program demonstrates matrix multiplication using Python.

def matrix_multiply(A, B):
    """
    Multiply two matrices A and B.

    A is an m x n matrix.
    B is an n x p matrix.
    The result C is an m x p matrix.
    """

    rows_A = len(A)
    cols_A = len(A[0])
    rows_B = len(B)
    cols_B = len(B[0])

    # Check whether the two matrices can be multiplied
    if cols_A != rows_B:
        raise ValueError("Matrix dimensions do not match for
multiplication.")
```

```python
    # Create a result matrix filled with zeros
    C = [[0 for _ in range(cols_B)] for _ in range(rows_A)]

    # Matrix multiplication:
    # C[i][j] = sum(A[i][k] * B[k][j])
    for i in range(rows_A):
        for j in range(cols_B):
            for k in range(cols_A):
                C[i][j] += A[i][k] * B[k][j]

    return C


def print_matrix(M):
    """Print a matrix row by row."""
    for row in M:
        print(row)


if __name__ == "__main__":
    A = [
        [1, 2],
        [3, 4]
    ]

    B = [
        [5, 6],
        [7, 8]
    ]

    print("Matrix A:")
    print_matrix(A)

    print("\nMatrix B:")
    print_matrix(B)

    C = matrix_multiply(A, B)

    print("\nResult of A x B:")
    print_matrix(C)

    print("\nExpected result:")
    print("[19, 22]")
    print("[43, 50]")
```

4.2 Execution Command

The Python program can be executed with the following command:

```
python matrix_multiply.py
```

4.3 Program Output

```
Matrix A:
[1, 2]
[3, 4]

Matrix B:
[5, 6]
[7, 8]

Result of A x B:
[19, 22]
[43, 50]

Expected result:
[19, 22]
[43, 50]
```

5. Algorithm Verification

To verify the correctness of the matrix multiplication algorithm, I used a small `2 × 2` matrix example. This example is simple enough to calculate manually.

The input matrices are:

```
A = [[1, 2],
     [3, 4]]

B = [[5, 6],
     [7, 8]]
```

The expected result is:

```
C = [[19, 22],
     [43, 50]]
```

Manual calculation:

```
C[0][0] = 1 × 5 + 2 × 7 = 19
C[0][1] = 1 × 6 + 2 × 8 = 22
C[1][0] = 3 × 5 + 4 × 7 = 43
C[1][1] = 3 × 6 + 4 × 8 = 50
```

The program output is:

```
[19, 22]
[43, 50]
```

The program output matches the manually calculated result. Therefore, the implementation is correct for this test case.

6. C Language Implementation and Performance Analysis

I did not complete the optional C language implementation because GCC was not installed in my current environment.

When I executed:

```
gcc --version
```

the result was:

```
bash: gcc: command not found
```

Therefore, this report only includes the required Python implementation.

7. Conclusion

In this assignment, I practiced using command line tools, writing Markdown documentation, and implementing a basic algorithm in Python.

Through the Python implementation, I learned how matrix multiplication works using three nested loops. I also verified the correctness of the algorithm by comparing the program output with a manually calculated result.

I also learned that different development environments may provide different command line tools. Since my environment was Windows with Git Bash, some Linux commands were unavailable, so I used PowerShell commands as alternatives.

Overall, this assignment helped me become more familiar with command line operations, Markdown documentation, Python programming, and basic algorithm verification.

8. References

- Python Documentation
- Git Documentation
- Markdown Guide
- Microsoft PowerShell Documentation

9. Appendix

The related files are available on my personal website:

- Python Source Code
- PDF Report